

FIAP · FASE 6 · NAVEGANDO PELO MUNDO DEVOPS

Cidades ESGInteligentes

EcoTrack API: inventário de emissões com pipeline CI/CD, Docker e deploy em staging e produção

Integrantes: Mateus Bhering

Repositório: github.com/mateusbhering/FIAP_Fase6_Desafio_Devops

Stack: Java 17 · Spring Boot 3.5 · PostgreSQL 16 · Docker · GitHub Actions

Data: setembro de 2026

Emissões de CO₂ de uma cidade, medidas e consolidadas

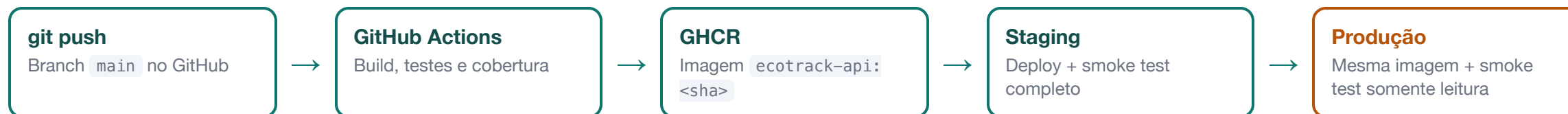
A **EcoTrack API** registra as emissões de gases de efeito estufa de cidades e organizações e as consolida em **toneladas de CO₂ equivalente (tCO₂e)**, seguindo os escopos do **GHG Protocol**. Isso dá à gestão pública um indicador ambiental (o “E” do ESG) acompanhável.

Escopo	Significado	Exemplo municipal
ESCOP0_1	Emissões diretas	Frota de ônibus a diesel
ESCOP0_2	Energia comprada	Iluminação pública
ESCOP0_3	Cadeia de valor	Coleta e aterro de resíduos

O que o desafio pedia, e onde está

Requisito	Entrega
Pipeline CI/CD	GitHub Actions: <code>ci-cd.yml</code> + <code>deploy.yml</code>
Build automático	Job <i>Build e testes</i> (Maven Wrapper)
Testes automatizados	11 testes, 95,7% de cobertura de linhas
Deploy staging + produção	Jobs encadeados, mesma imagem nos dois
Dockerfile	Multi-stage, não-root, healthcheck
Orquestração	<code>docker-compose.yml</code> : API + PostgreSQL
Volumes, env, redes	<code>db-data</code> , <code>app-logs</code> · <code>.env</code> por ambiente · rede interna

Do commit ao ambiente em execução



Cada ambiente = um projeto Docker Compose (ecotrack-staging · ecotrack-prod)

rede frontend (exposta)

app

Spring Boot · porta 8081 (stg) / 8080 (prd)
usuário não-root · healthcheck

volume app-logs

rede backend (internal: true, sem saída externa)

app ⇌ db

JDBC `jdbc:postgresql://db:5432/...`

db

PostgreSQL 16 · sem porta publicada · healthcheck `pg_isready`

volume db-data

Camadas da aplicação

- **controller**: REST + validação (Bean Validation), erros no padrão RFC 7807
- **service**: regras e consolidação em tCO₂e
- **repository**: Spring Data JPA
- **Flyway** versiona o schema; o Hibernate só valida (`ddl-auto: validate`)
- **Actuator**: `/health` , `/readiness` , `/info` (versão e ambiente)
- **Perfis**: `staging` (debug, Swagger) · `prod` (log WARN, sem Swagger, health sem detalhes)

Quatro etapas encadeadas, com um gate entre cada uma

1 Build e testes

`./mvnw verify` compila e roda os 11 testes (unitários, web e integração) e a cobertura JaCoCo. Publica o resumo dos testes e os relatórios e o `.jar` como artefatos.

2 Imagem Docker

Buildx com cache do GitHub Actions. Push para o GHCR com as tags `<sha>` (imutável) e `main`.

3 Deploy staging

Environment `staging`. `deploy.sh` sobe o Compose, espera a readiness e roda o smoke test completo: cria, consulta, consolida e apaga um registro.

4 Deploy produção

Environment `production`, que aceita aprovação manual. Implanta a **mesma imagem** e roda o smoke test somente leitura.

Lógica

- **Gatilhos:** push na `main` roda tudo. *Pull request* roda só a etapa 1. Commits só de documentação não disparam.
- **Gates:** teste falhou, não há imagem. Staging falhou, não há produção.
- **Build once, deploy many:** produção recebe exatamente o artefato validado em staging.
- **Rollback automático:** se a nova versão não fica pronta, o script volta a anterior.
- **Concorrência:** dois deploys nunca rodam ao mesmo tempo (`concurrency`).

Segredos e alvo do deploy

- A senha do banco vem dos **secrets de cada Environment**. No repositório ficam só configurações não sensíveis (`deploy/*.env`).
- **Servidor via SSH:** com `DEPLOY_SSH_*` configurados, envia o Compose e os scripts para uma VM com Docker.
- **Runner do GitHub** (configuração atual): o ambiente sobe no próprio runner com o mesmo Compose e o mesmo script. Valida ponta a ponta, e o ambiente é efêmero.
- **Dependabot** mantém as dependências, as actions e as imagens atualizadas.

Dockerfile multi-stage: build pesado, runtime enxuto

```
# Etapa 1: build
FROM maven:3.9-eclipse-temurin-17 AS build
COPY pom.xml .
RUN --mount=type=cache,target=/root/.m2 mvn dependency:go-offline
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn package -DskipTests \
    && java -Djarmode=tools -jar target/ecotrack-api.jar \
        extract --layers --launcher --destination target/extracted

# Etapa 2: runtime
FROM eclipse-temurin:17-jre
RUN groupadd --system ecotrack && useradd --system ... ecotrack
COPY --from=build .../dependencies/ ./
COPY --from=build .../spring-boot-loader/ ./
COPY --from=build .../snapshot-dependencies/ ./
COPY --from=build .../application/ ./
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 ..."
USER ecotrack
EXPOSE 8080
VOLUME ["/app/logs"]
HEALTHCHECK CMD curl -fsS .../actuator/health/readiness ...
ENTRYPOINT ["java",
    "org.springframework.boot.loader.launch.JarLauncher"]
```

Estratégia	Benefício
Multi-stage	A imagem final não leva Maven, JDK nem código-fonte
Cache de dependências	Mudar o código não baixa as bibliotecas de novo
Camadas do Spring Boot	Bibliotecas (60 MB) separadas do código (188 kB); um novo deploy transfere quase nada
Usuário não-root	Menor impacto em caso de invasão
Healthcheck na readiness	O Compose sabe quando a API está pronta de fato
MaxRAMPercentage	A JVM respeita o limite de memória do container
APP_VERSION = SHA	Rastreabilidade em <code>/actuator/info</code>
Base multi-arquitetura	Roda em amd64 (CI) e arm64 (Apple Silicon)

Um arquivo Compose para três ambientes

Recurso	Configuração
Serviços	db (PostgreSQL 16) e app, com depends_on: service_healthy e restart: unless-stopped
Volumes	db-data (dados persistentes), app-logs (logs)
Redes	backend interna (app ⇌ banco), frontend (expõe só a API)
Variáveis	.env local · deploy/staging.env · deploy/production.env · senha via secret, obrigatória (\${POSTGRES_PASSWORD:?})

	Staging	Produção
Projeto (-p)	ecotrack-staging	ecotrack-prod
Porta	8081	8080
Perfil Spring	staging	prod
Smoke test	completo (escrita)	somente leitura

```
# Local
cp .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs -f app
docker compose down # mantém volumes

# Build da imagem
docker build -t ecotrack-api:local .

# Ambientes (o mesmo script do pipeline)
export APP_IMAGE=ecotrack-api:local SKIP_PULL=true
POSTGRES_PASSWORD=*** ./scripts/deploy.sh staging
POSTGRES_PASSWORD=*** ./scripts/deploy.sh production

# Por baixo, o deploy.sh executa:
docker compose -p ecotrack-staging \
  --env-file deploy/staging.env up -d --no-build

# Verificação pós-deploy
./scripts/smoke-test.sh http://localhost:8081
./scripts/smoke-test.sh http://localhost:8080 --read-only

# Testes
./mvnw verify
```

ecotrack-api: 162 MB, não-root, com healthcheck

```
Imagem Docker criada – ecotrack-api

$ docker image ls ecotrack-api
IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
ecotrack-api:local   a49afbba3eee      562MB         162MB           U

$ docker image inspect ecotrack-api:local --format '...'
Usuario: ecotrack
Porta: 8080/tcp
Volumes: /app/logs
Entrypoint: ["java","org.springframework.boot.loader.launch.JarLauncher"]
Healthcheck: ["CMD-SHELL","curl -fsS http://localhost:8080/actuator/health/readiness | grep -q '\"status\": \"UP\"' || exit 1"]
Arquitetura: linux/arm64
Camadas: 12

$ docker history ecotrack-api:local --format 'table {{.Size}}\t{{.CreatedBy}}' | head -12
SIZE      CREATED BY
0B        ENTRYPOINT ["java" "org.springframework.boot...
0B        HEALTHCHECK {Test:[CMD-SHELL curl -fsS http:...
0B        VOLUME [/app/logs]
0B        EXPOSE [8080/tcp]
0B        USER ecotrack
0B        ENV APP_VERSION=ccf68d9 LOG_DIR=/app/logs JA...
0B        ARG APP_VERSION=ccf68d9
188kB     COPY --chown=ecotrack:ecotrack /build/target...
4.1kB     COPY --chown=ecotrack:ecotrack /build/target...
696kB     COPY --chown=ecotrack:ecotrack /build/target...
60.4MB    COPY --chown=ecotrack:ecotrack /build/target...
```

No pipeline, a mesma imagem é publicada em ghcr.io/mateusbhering/ecotrack-api com a tag do commit. A camada da aplicação tem 188 kB; as dependências (60 MB) ficam em cache.

GitHub Actions: build, testes, imagem, staging e produção

Platform Solutions Resources Open Source Enterprise Pricing

Search

Sign in

Sign up

mateusbhering / FIAP_Fase6_Desafio_Devops

Public

Notifications

Fork 0

Star 0

<> Code

Issues

Pull requests 2

Actions

Projects

Security and quality

Insights

← CI/CD

Atualiza actions para versoes atuais, Maven 3.9.16 e titulo da OpenAPI #13

Sign in to view logs

Summary

All jobs

Build e testes

Build e push da imagem Docker

Staging

Producao

Run details

Usage

Workflow file

Triggered via push 6 minutes ago

Status

Total duration

Artifacts

mateusbhering pushed -o- ccf68d9 main

Success

4m 14s

3

ci-cd.yml

on: push

Build e testes 50s

Build e push da imagem... 2m 6s

Stagin... / Deploy em staging 36s

Pr... / Deploy em production 28s

Execução pública em github.com/mateusbhering/FIAP_Fase6_Desafio_Devops/actions

Deploys anotados e artefatos gerados pelo pipeline

Annotations

6 notices

1

Build e testes

"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runn...>

Show more

1

Build e push da imagem Docker

"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runn...>

Show more

1

Staging / Deploy em staging

DEPLOY_SSH_HOST nao configurado para 'staging': deploy no runner (ambiente efemero).

1

Staging / Deploy em staging

"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runn...>

Show more

1

Producao / Deploy em production

DEPLOY_SSH_HOST nao configurado para 'production': deploy no runner (ambiente efemero).

1

Producao / Deploy em production

"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runn...>

Show more

Artifacts

Produced during runtime

Name	Size	Digest
ecotrack-api-jar	51.6 MB	sha256:1511c4a98952acb0cede0b9042542ae8f0d... 🔗
mateusbhering-FIAP_Fase6_Desafio_Devo ps-2L5LB5.dockerbuild	65.5 KB	sha256:7b6ed0c09bcf8770f5dff8004d95241c4e3... 🔗
relatorios-testes	252 KB	sha256:f45013d3107a56d30e3da3ea867e2f6f975... 🔗

Cada deploy registra o alvo usado. O pipeline publica o `ecotrack-api-jar`, os relatórios de testes e cobertura (`relatorios-testes`) e o registro do build Docker.

Cidades ESGInteligentes · EcoTrack API · Desafio DevOps

Deploys registrados nos environments staging e production

GitHub – deploys nos environments staging e production (API pública /deployments + /statuses)

```
$ GET https://api.github.com/repos/mateusbhering/FIAP_Fase6_Desafio_Devops/deployments (tabela resumida)
```

AMBIENTE	COMMIT	CRIADO EM (UTC)	STATUS	EXECUCAO
production	ccf68d9	2026-09-19T15:47:44Z	success	mateusbhering/FIAP_Fase6_Desafio_Devops/actions/runs/35452729419/job/105923092221
staging	ccf68d9	2026-09-19T15:47:05Z	success	mateusbhering/FIAP_Fase6_Desafio_Devops/actions/runs/35452729419/job/105923004568
production	379feef	2026-09-19T15:38:36Z	success	mateusbhering/FIAP_Fase6_Desafio_Devops/actions/runs/35452205960/job/105921846720
staging	379feef	2026-09-19T15:38:02Z	success	mateusbhering/FIAP_Fase6_Desafio_Devops/actions/runs/35452205960/job/105921767812

Dados da API pública do GitHub: cada execução do pipeline registrou um deploy em *staging* e outro em *production*, todos com status **success**.

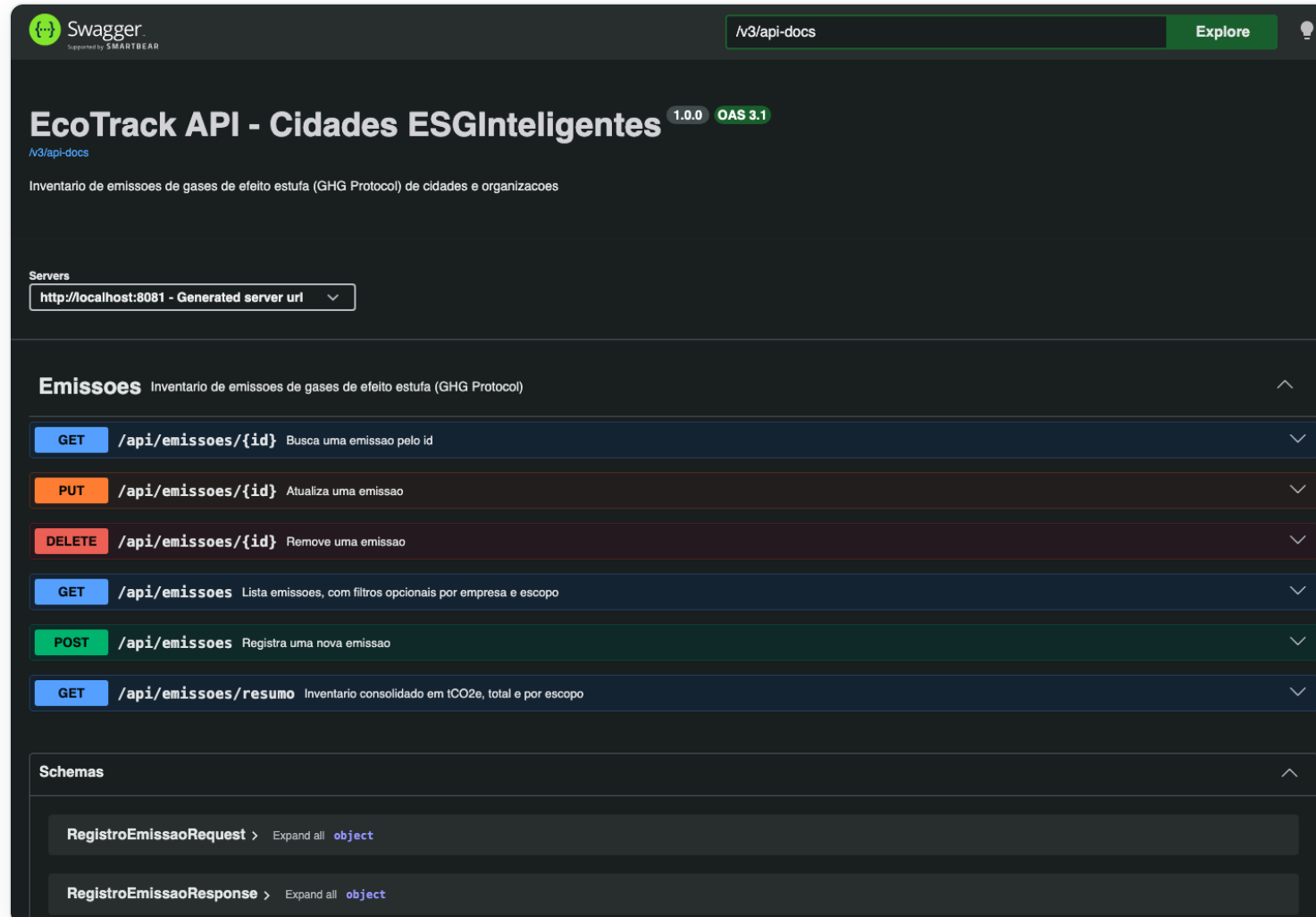
11 testes aprovados, 95,7% de cobertura de linhas

```
Testes automatizados – trechos da saída do Maven + cobertura JaCoCo (docs/evidencias/testes-automatizados.txt)

$ ./mvnw -B -ntp verify
[INFO] --- jacoco:0.8.15:prepare-agent (default) @ ecotrack-api ---
[INFO] Running br.com.fiap.ecotrack.controller.RegistroEmissaoControllerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.711 s -- in br.com.fiap.ecotrack.controller.RegistroEmissaoControllerTest
[INFO] Running br.com.fiap.ecotrack.EcoTrackIntegrationTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.008 s -- in br.com.fiap.ecotrack.EcoTrackIntegrationTest
[INFO] Running br.com.fiap.ecotrack.service.RegistroEmissaoServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.094 s -- in br.com.fiap.ecotrack.service.RegistroEmissaoServiceTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.15:report (report) @ ecotrack-api ---
[INFO] BUILD SUCCESS
[INFO] Total time: 22.320 s
Cobertura de linhas (JaCoCo): 95.7% (112/117 linhas)
ecotrack      EcoTrackApplication      1/3
exception     GlobalExceptionHandler      11/11
exception     RecursoNaoEncontradoException 2/2
dto           RegistroEmissaoResponse      9/9
dto           ResumoEmissoesResponse      1/1
dto           RegistroEmissaoRequest       1/1
controller    RegistroEmissaoController    14/15
model         RegistroEmissao              24/24
model         EscopoEmissao                4/4
service       RegistroEmissaoService       45/47
```

Testes unitários (service), de camada web (controller com MockMvc) e de integração (contexto completo + Flyway + banco). No pipeline, a mesma execução roda no job *Build e testes*.

Staging funcionando: documentação da API



Swagger UI habilitado no perfil `staging` (<http://localhost:8081>).

Staging funcionando: chamadas à API

STAGING — http://localhost:8081

```
$ curl -s localhost:8081/actuator/info | jq .app
```

```
{
  "name": "EcoTrack API",
  "environment": "staging",
  "version": "local"
}
```

```
$ curl -s localhost:8081/actuator/health | jq -c '{status, db: .components.db}'
```

```
{"status":"UP","db":{"status":"UP","details":{"database":"PostgreSQL","validationQuery":"isValid()"}}}
```

```
$ curl -s 'localhost:8081/api/emissoes?empresa=Prefeitura%20de%20Sao%20Paulo&escopo=ESCOPO_1' | jq -c '.[0]'
```

```
{"id":1,"empresa":"Prefeitura de Sao Paulo","escopo":"ESCOPO_1","fonte":"Frota de onibus a diesel","quantidadeCo2eKg":185200.400,"dataReferencia":"2026-06-30","criadoEm":"2026-09-19T15:35:43.161840Z"}
```

```
$ curl -s 'localhost:8081/api/emissoes/resumo?empresa=Prefeitura%20de%20Sao%20Paulo' | jq .
```

```
{
  "empresa": "Prefeitura de Sao Paulo",
  "quantidadeRegistros": 3,
  "totalTCo2e": 524.371,
  "totalPorEscopoTCo2e": {
    "ESCOPO_1": 185.200,
    "ESCOPO_2": 98.300,
    "ESCOPO_3": 240.870
  }
}
```

/actuator/info reporta environment: staging ; health mostra o PostgreSQL conectado; inventário da cidade consolidado em tCO₂e por escopo.

Produção funcionando (porta 8080, perfil prod)

PRODUÇÃO — http://localhost:8080

```
$ curl -s localhost:8080/actuator/info | jq .app
{
  "name": "EcoTrack API",
  "environment": "production",
  "version": "local"
}

$ curl -s localhost:8080/actuator/health # detalhes ocultos em producao
{"status":"UP","groups":["liveness","readiness"]}

$ curl -s 'localhost:8080/api/emissoes?escopo=ESCOPO_2' | jq -c '[]'
{"id":2,"empresa":"Prefeitura de Sao Paulo","escopo":"ESCOPO_2","fonte":"Iluminacao publica","quantidadeCo2eKg":98300.000,"dataReferencia":"2026-06-30","criadoEm":"2026-09-19T15:35:43.283066Z"}

$ curl -s localhost:8080/api/emissoes/resumo | jq .
{
  "empresa": "TODAS",
  "quantidadeRegistros": 3,
  "totalTCo2e": 524.371,
  "totalPorEscopoTCo2e": {
    "ESCOPO_1": 185.200,
    "ESCOPO_2": 98.300,
    "ESCOPO_3": 240.870
  }
}

$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8080/swagger-ui/index.html # Swagger desligado no perfil prod
404
```

Ambiente reportado como `production`, health sem detalhes internos e Swagger desativado (404), como esperado em produção.

Containers, volumes, redes e variáveis de ambiente

```
Docker – containers, volumes e redes dos dois ambientes

$ docker ps --filter name=ecotrack --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
NAMES                IMAGE                STATUS              PORTS
ecotrack-prod-app-1   ecotrack-api:local   Up 9 minutes (healthy)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
ecotrack-staging-app-1 ecotrack-api:local   Up 9 minutes (healthy)  0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
ecotrack-prod-db-1    postgres:16-alpine   Up 18 minutes (healthy) 5432/tcp
ecotrack-staging-db-1 postgres:16-alpine   Up 18 minutes (healthy) 5432/tcp

$ docker volume ls --filter name=ecotrack
DRIVER    VOLUME NAME
local     ecotrack-prod_app-logs
local     ecotrack-prod_db-data
local     ecotrack-staging_app-logs
local     ecotrack-staging_db-data

$ docker network ls --filter name=ecotrack
NETWORK ID    NAME                DRIVER    SCOPE
fc78f4d2e36b  ecotrack-prod_backend bridge    local
8710617d2585  ecotrack-prod_frontend bridge    local
619554789d65  ecotrack-staging_backend bridge    local
61095dbd665d  ecotrack-staging_frontend bridge    local

$ docker network inspect ecotrack-prod_backend --format 'rede={{.Name}} internal={{.Internal}} containers={{range .Containers}}{{.Name}} {{end}}'
rede=ecotrack-prod_backend internal=true containers=ecotrack-prod-db-1 ecotrack-prod-app-1

$ docker exec ecotrack-prod-app-1 env | grep -E '^(SPRING_PROFILES_ACTIVE|APP_ENVIRONMENT|SPRING_DATASOURCE_URL)= ' | sort
APP_ENVIRONMENT=production
SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/ecotrack
SPRING_PROFILES_ACTIVE=prod

$ docker exec ecotrack-prod-app-1 id
uid=999(ecotrack) gid=999(ecotrack) groups=999(ecotrack)
```

Quatro containers *healthy*; volumes e redes isolados por ambiente; rede backend `internal=true`; variáveis injetadas pelo Compose; processo rodando como usuário não-root.

Smoke tests aprovados em staging e produção

```
Smoke tests pós-deploy – staging e produção

$ EXPECTED_ENV=staging ./scripts/smoke-test.sh http://localhost:8081
Smoke test em http://localhost:8081
[OK] health UP
[OK] ambiente reportado = staging
[OK] GET /api/emissoes -> 200
[OK] GET /api/emissoes/resumo -> 200
[OK] POST invalido -> 400
[OK] POST /api/emissoes cria registro
[OK] GET /api/emissoes/5 -> 200
[OK] resumo da empresa soma 1.235 tCO2e
[OK] DELETE /api/emissoes/5 -> 204
Smoke test OK.

$ EXPECTED_ENV=production ./scripts/smoke-test.sh http://localhost:8080 --read-only
Smoke test em http://localhost:8080
[OK] health UP
[OK] ambiente reportado = production
[OK] GET /api/emissoes -> 200
[OK] GET /api/emissoes/resumo -> 200
[OK] POST invalido -> 400
Smoke test OK.
```

Staging: teste completo, com escrita e limpeza. Produção: somente leitura. São os mesmos scripts executados pelo pipeline após cada deploy.

Rollback automático de uma versão quebrada

Rollback automático – saída registrada em docs/evidencias/rollback-automatico.txt

```
$ APP_IMAGE=ecotrack-api:quebrada DEPLOY_TIMEOUT=30 ./scripts/deploy.sh staging
==> Deploy de ecotrack-api:quebrada em staging (projeto ecotrack-staging, porta 8081)
    Versao atual: ecotrack-api:local
Aguardando http://localhost:8081/actuator/health/readiness (timeout 30s)...
!! A nova versao nao ficou pronta. Logs da aplicacao:
app-1 | Falha simulada na inicializacao
app-1 | Falha simulada na inicializacao
app-1 | Falha simulada na inicializacao
app-1 | Falha simulada na inicializacao
app-1 | Falha simulada na inicializacao
app-1 | Falha simulada na inicializacao
app-1 | Falha simulada na inicializacao
app-1 | Falha simulada na inicializacao
app-1 | Falha simulada na inicializacao
!! Rollback para ecotrack-api:local
Aguardando http://localhost:8081/actuator/health/readiness (timeout 30s)...
!! Rollback concluido; staging segue na versao anterior

Imagem em execucao apos o rollback: ecotrack-api:local
Health: {"status":"UP"}
```

A readiness probe detecta que a nova versão não subiu; o `deploy.sh` volta sozinho para a imagem anterior e o job falha, bloqueando a promoção.

Persistência de dados no volume db-data

Persistência – dados sobrevivem à recriação dos containers

```
$ cat docs/evidencias/persistencia-volume.txt
# Persistencia: dados sobrevivem a remocao dos containers (volume db-data)
Registros antes: {"empresa":"FIAP","quantidadeRegistros":3,"totalTCo2e":70.421,"totalPorEscopoTCo2e":{"ESCOPO_1":18.250,"ESCOPO_2":42.300,"ESCOPO_3":9.870}}
$ docker compose -p ecotrack-staging down # remove containers e redes, mantem volumes
$ ./scripts/deploy.sh staging
==> Deploy de ecotrack-api:local em staging (projeto ecotrack-staging, porta 8081)
==> staging no ar com ecotrack-api:local
Registros depois: {"empresa":"FIAP","quantidadeRegistros":3,"totalTCo2e":70.421,"totalPorEscopoTCo2e":{"ESCOPO_1":18.250,"ESCOPO_2":42.300,"ESCOPO_3":9.870}}
```

Os containers e as redes são removidos com `docker compose down` e recriados pelo deploy; os registros continuam lá.

Desafios encontrados e como foram resolvidos

Desafio	Solução
A imagem <code>eclipse-temurin:17-jre-alpine</code> não tem versão ARM64: o build falhava no Mac com Apple Silicon (<i>no match for platform in manifest</i>)	Troca para <code>eclipse-temurin:17-jre</code> (multi-arquitetura) e ajuste dos comandos de usuário e healthcheck para a base Ubuntu
O Docker travava ao montar a pasta do projeto (Desktop do macOS) como <i>bind mount</i>	O Compose passou a usar só volumes nomeados ; o build envia o contexto sem depender de montagem de pasta
Maven não instalado na máquina de desenvolvimento	Maven Wrapper (<code>./mvnw</code>): a mesma versão do Maven no computador e no CI, sem instalação
Sem servidor em nuvem disponível para staging e produção	Workflow de deploy com dois alvos: SSH para uma VM ou o runner do GitHub, usando o mesmo Compose e o mesmo script
Evitar que uma versão quebrada derrube um ambiente	Espera da <i>readiness probe</i> , rollback automático , smoke tests e gate staging → produção
Testes de integração sem depender de um PostgreSQL no CI	H2 em modo PostgreSQL, rodando as mesmas migrations Flyway de produção
Senha do banco sem ir para o repositório	Secrets por Environment, senha efêmera e mascarada no runner, Compose que não sobe sem a senha
Primeira execução com avisos de actions obsoletas (Node 20) e PRs do Dependabot com saltos de versão principal	Actions atualizadas para as versões atuais; Dependabot configurado para ignorar migrações de versão principal (Spring Boot 4, Java 24)

Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☒
Dockerfile funcional	☒
docker-compose.yml ou arquivos Kubernetes	☒
Pipeline com etapas de build, teste e deploy	☒
README.md com instruções e prints	☒
Documentação técnica com evidências (PDF ou PPT)	☒
Deploy realizado nos ambientes staging e produção	☒

Repositório e pipeline públicos: github.com/mateusbhering/FIAP_Fase6_Desafio_Devops